Weighted Word Embeddings As A Way To Reduce Information Loss After Matrix Factorization

Xiaodong Liu
Graduate School of Information Science and Technology
Rafal Rzepka, Kenji Araki
Faculty of Information Science and Technology
Hokkaido University
Sapporo, Japan
{xiaodongliu,rzepka,araki}@ist.hokudai.ac.jp

Abstract

The Vector Space Model (VSM) is a distributional model that can represent texts as points in a space; the points in the space that are close to each other can be considered as semantically similar. On the other hand, word embedding is another distributional model that can embed words in a space; semantic similarity can also be evaluated by how close two points are in the space. In this paper, we propose a novel method to project weighted word embeddings on latent space consisting of singular vectors without reweighting before matrix factorization, as a new approach to decreasing information loss after matrix factorization and improving distributional sentences similarity. As the first foray into this approach, instead of complex deep learning neural networks, we just utilized a simple two-layer feed-forward neural network to implement our proposed method. Microsoft Research Paraphrase Corpus was used by us to examine whether the proposed method can measure semantic similarity between two sentences. Regarding the performance, our proposed method outperforms many previously published distributional methods and some methods that utilize complex neural networks.

1 Introduction

According to the survey written by (Turney and Pantel, 2010), the VSM was developed for the SMART information retrieval system (Salton, 1971), which pioneered many of the concepts that are used in modern search engines (Manning et al., 2008). One form of the VSM is term-document matrix, which contains the documents representing phrases, sentences, or texts, and terms representing all the single words of those documents. The key idea of the VSM is to represent all the documents in a term-document matrix as points in a space (distributional representations). The points that are close in the space can be considered as semantically similar, whereas the points that are far apart from each other are semantically distant.

The vector representation of each document is Bag-of-Words. The frequencies of words in documents, to some extent, can determine how relevant they are. However, when the size of the matrix X ($M \times N$) is enormous, it will consume a considerable memory footprint. Moreover, the documents used in our experiment are short texts (sentences), which makes the matrix sparse. To address this limitation, matrix factorization was introduced, and one standard way to perform this mathematical operation is Singular Vector Decomposition (SVD) (Deerwester et al., 1990), which is the essential part of LSA (Latent Semantic Analysis) (Landauer and Dumais, 1997). SVD decomposes the matrix X ($M \times N$) into the product of three matrices U ($M \times K$) $\cdot \sum (K \times K) \cdot V^T$ ($K \cdot N$), in which U is a left singular matrix, $\sum$ is a diagonal matrix of singular values, V^T is a right singular matrix, where $K \ll M$ or N. Proximity in the induced latent space has been shown to correlate with semantic similarity (Mihalcea et al., 2006).

Word embedding models have evolved a lot ever since the advent of word2vec proposed by (Mikolov et al., 2013), which uses continuous Bag-of-Words or skipgram model to train a corpus with a local sliding window over each sentence of the corpus. As a result, words that tend to co-occur in different sentences are close in the embedding space. Global Vectors (GloVe) (Pennington et al., 2014), instead of using neural network to train a corpus based on sentence-level, exploits the global statistical information regarding word co-occurrences. Word co-occurrence matrix provides important information as to the probability that two words co-ocur in the entire corpus. For example, with a symmetric window size set at 3, if the word “ice” appears 10 times in a corpus, then 60 candidates of

co-occurred words are counted. Within the candidates, if the word "solid" appears 2 times, then the co-occurrence probability for "solid" in the global context of "ice" is $\frac{2}{60}$. Based on the global statistical information, the word embeddings trained by the algorithm of GloVe are more related globally. An extension of word2vec is fastText (Bojanowski et al., 2017), the design of which takes into consideration the morphology of words. In fastText, each word is represented as a bag of character n-grams. A vector representation is associated to each character n-gram; words being represented as the sum of these representations. For example, the word "where" with windows size set at 3, the fastText representation of this word is $\langle wh, whe, her, ere, re\rangle$, where the angular brackets indicate the beginning and end of the word. This approach can also help understand suffixes and prefixes.

The above mentioned distributional semantic similarities generated by the VSM or word embedding are mainly derived from considerations such as frequencies of words and word co-occurrences: information contained in sentences is processed from different perspectives. They have their own advantages and disadvantages. The purpose of this work is to see whether the distributional property of word embedding can be used to address the limitation of the VSM and thus improve distributional semantic similarity of sentences, the details of which are described in the next section. The results of two preliminary experiments used for word embedding selection and one criterion of our weighting scheme are described in Section 3. Concrete introduction of our proposed method is described in Section 4. The features related to our proposed method are described in Section 5.

Microsoft Research Paraphrase Corpus (MSRP) (Dolan et al., 2004) comprises 5801 paraphrase sentence pairs, in which 4076 sentence pairs are grouped into training dataset (2753 pairs are labeled as paraphrase) while the rest of 1725 sentence pairs are grouped into test dataset (1147 pairs are labeled as paraphrase). The results of features-based paraphrase identification are shown in Section 6.

2 Solution to Information Loss

Matrix factorization is considered as an improvement to distributional model with respect to semantic similarity — the simplest way to improve information retrieval performance is to limit the number of vector components (Turney and Pantel, 2010); however, a considerable amount of information can be lost after matrix factorization. To address this limitation, there are two typical approaches: to improve mathematical operation of matrix factorization; to reweight original term-document matrix (Bag-of-Words vector representation for documents) before matrix factorization.

The way to improve matrix factorization is to smooth term-document matrix in an elegant manner so that the factorized matrix can be used for particular tasks. Some classic matrix smoothing processes have been discovered since the advent of SVD such as Nonnegative Matrix Factorization (NMF) (Lee and Seung, 1999), which ensures all elements in the factorized matrix are nonnegative; probability-based Latent Dirichlet Allocation (LDA) (Blei et al., 2003), which factorizes the term-document matrix into document-topics distribution and topic-terms distribution.

The main purpose of weighting schemes is to give more weight to discriminative words like rare words, and less weight to common words like "the". The most popular way to perform this trick is tf-idf (term frequency $\times$ inverse document frequency). (Salton and Buckley, 1988) defined a large family of tf-idf weighting functions and evaluated them on information retrieval tasks. There are some alternatives to tf-idf such as Pointwise Mutual Information (PMI) (Church and Hanks, 1990) or length normalization combined with tf-idf (Singhal et al., 1996). (Ji and Eisenstein, 2013) considered that if the factorization does not take into account labeled data about semantic similarity, important information can be lost. Therefore, a special weighting scheme, TF-KLD, was utilized, which used Kullback-Leibler divergence to measure distance between two Bernoulli distributions: given a feature appears in the second sentences of paraphrase sentence pairs or non-paraphrase sentence pairs, what is the probability that it also appears in the first sentences. For example, if the two Bernoulli distributions for the feature "not" are [0.66, 0.34] and [0.31, 0.69], their KL-divergence is 0.25; the value is 0 when two distributions are identical, which means the feature is not discriminative at all. However, this weighting scheme is effective only when a corpus is related to binary classification.

In this paper, focus is shed on neither matrix factorization nor term-document weighting schemes. Instead, we improve distributional semantic similarity by shifting focus to weighted word embeddings,

which can be considered as a way to decrease information loss after matrix factorization. As we stated in the previous section, since the distributional properties of two distributional models are different, the advantage of one could potentially offset the disadvantage of another. Hence, we adopt mapping mechanism of neural network to combine them together. Regarding the weighting schemes for word embedding, syntactic elements play different roles as characteristics of corpora can vary considerably, so we weight word embeddings according to their significance in syntactic roles.

3 Preliminary Experiment

In this section, the results provided by two preliminary experiments are conducive to the further experiment regarding our proposed method. The result of word embeddings selection determines which pretrained word embedding model is used in the further experiment, and the result of Jaccard distance-based paraphrase identification is used as one criterion of our weighting scheme.

In this paper, all experiments, including the preliminary experiments, use the Natural Language Toolkit (NLTK) [1] to perform relevant operations regarding natural language processing.

3.1 Word Embedding Selection

We collected 100 pairs of synonyms from WordNet (Miller, 1995) in alphabetical order of first word of a pair from A to Z: "accomplish" to "achieve", "pretty" to "beautiful", and so on. The cosine similarity between each pair's word embeddings of three different models is computed: word2vec[2], GloVe[3], and fastText[4]; the mean and standard deviation of cosine similarity are illustrated in Table 1. Since fastText had the best performance, we decided to use it as pretrained word embeddings.

3.2 One Criterion of Our Weighting Scheme

Jaccard distance (Jaccard, 1912) has also been used by other researchers, e.g. (Xu et al., 2013), for paraphrase identification. Equation 1 explains that the more elements two sets intersect, the smaller their Jaccard distance is.

$$Jdistance(A,B) = \frac{|A \cup B| - |A \cap B|}{|A \cup B|} \quad (1)$$

[1] https://www.nltk.org/
[2] GoogleNews-vectors-negative300.bin
[3] glove.840B.300d.txt
[4] crawl-300d-2M-subword.vec

Pretrained Model	Mean	Std
word2vec	0.54	0.18
GloVe	0.65	0.15
fastText	0.70	0.12

Table 1: Mean and standard deviation of cosine similarities between 100 synonym pairs' word embeddings of three pretrained models.

In our experiment, set A is the tokenized first sentence in lower case of a sentence pair, while set B is the tokenized second sentence in lower case of a sentence pair (this operation is also performed in our proposed method). Jaccard distance is computed on all sentence pairs belonging to the training dataset, and the distributions corresponding to the paraphrase and non-paraphrase sentence pairs are illustrated in Figure 1.

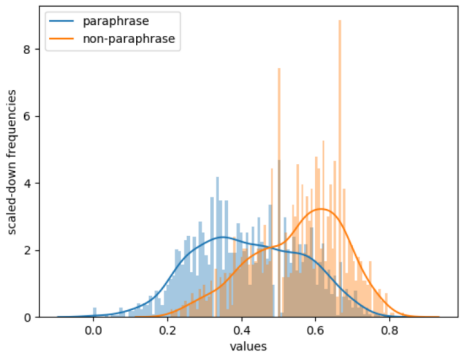


Figure 1: Jaccard distance distributions of paraphrase and non-paraphrase sentence pairs belonging to the training dataset of the MSRP corpus; the mean and standard deviation for paraphrase and non-paraphrase are [0.42, 0.14] and [0.55, 0.12] respectively.

Extrapolating from Figure 1, we set eleven thresholds (T) to identify all sentence pairs belonging to the test dataset. The thresholds are introduced to alleviate the problem of the uneven dataset: there are almost twice more paraphrase pairs than non-paraphrase pairs in the MSRP corpus. The result is shown in Table 2.

As shown in Table 2, Jaccard distance works for paraphrase identification to some extent; the highest accuracy reaches 73.39% associated with the F-score 81.78%, and the highest F-score reaches 81.97% associated with the accuracy 73.33%, which outperforms the baseline significantly excerpted from (Mihalcea et al., 2006): accuracy 65.4% and F-score 75.3%. The threshold of 0.6 is selected for one criterion of our weighting scheme as F increases drastically at 0.6 surpassing 81%.

T	Acc.(%)	Prec.(%)	Rec.(%)	F(%)
0.55	70.03	**77.49**	77.42	77.45
0.56	70.84	77.2	79.69	78.42
0.57	71.25	76.92	81.08	78.95
0.58	72.41	76.61	84.22	80.23
0.59	72.81	76.4	85.53	80.71
0.6	73.27	75.41	88.75	81.54
0.61	**73.39**	75.07	89.8	81.78
0.62	73.33	74.45	91.19	**81.97**
0.63	72.75	73.43	92.5	81.87
0.64	72.41	72.65	93.81	81.89
0.65	72	71.99	**94.77**	81.82

Table 2: Results of identifying paraphrase by Jaccard distance (if J-value <= threshold; then paraphrase).

4 Proposed Method

In this section, first we describe our proposed method by explaining each word-embedding component that is considered as information pertaining to literal semantics, and what sort of vector representations that we use for latent semantics. Then how to weight word-embedding components is described in a separate subsection. The final subsection describes the details about the neural network. The mapping flowchart of our proposed method is illustrated in Figure 2.

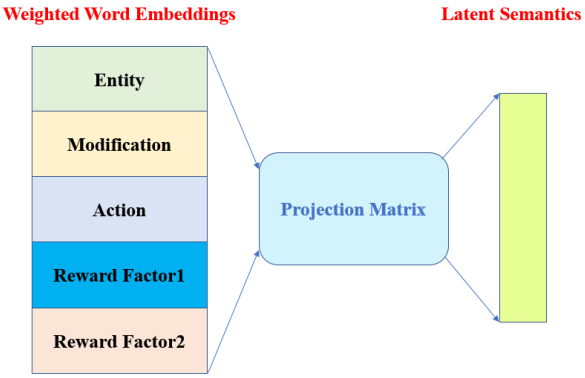


Figure 2: Mapping flowchart: Entity, Modification, and Action are weighted word embedding components corresponding to different syntactic elements; reward factors are weighted vectors with the same length as word embedding; latent semantics is the vector representation of latent semantics; projection matrix contains all parameters of the neural network.

4.1 Latent Semantics

Firstly, we used all sentences belonging to the training dataset, regardless of paraphrase pairs or non-paraphrase pairs, to form the term-document matrix using the Scikit-learn tool (Pedregosa et al., 2011). Secondly, the term-document matrix was factorized using SVD (to compare with (Guo and Diab, 2012), latent dimensionality K = 100), and then as we mentioned in Section 1, a right singular matrix was generated, in which each column is a right singular vector representing the latent semantics of a sentence. Thirdly, we normalized the right singular matrix by dividing all its elements (singular values) by the biggest singular value in the matrix plus 0.1 (e.g. 5.5 + 0.1 = 5.6) as the activation function used in our feedforward neural network is tanh. After normalization, the biggest singular value in the matrix is less than 1 and bigger than -1.

After obtaining all sentences' singular vectors, we generate their vector representations of latent semantics. As for non-paraphrase sentence pairs, their vector representations of latent semantics are the generated singular vectors; for example, if the singular vectors of a sentence S1 and a sentence S2 of a sentence pair are (0.9, 0.8, 0.7) and (0.7, 0.8, 0.9) respectively, then the two singular vectors are their latent semantics respectively. However, if S1 and S2 are a paraphrase sentence pair, then they share a vector representation of latent semantics by averaging the elementwise addition of their singular vectors, in this case {(0.9, 0.8, 0.7) + (0.7, 0.8, 0.9)} / 2 = (0.8, 0.8, 0.8).

4.2 Word-Embedding Components

There are three word-embedding components and two reward components: Entity, Modification, Action, Reward Factor1, and Reward Factor2. The concatenated five components are considered as overall information about literal semantics. In this subsection, we introduce the three word-embedding components; the description of two reward components are given in the next subsection.

All part-of-speech tags (POS-tags) of sentences used in our experiment are provided by the NLTK. In a sentence, entity component comprises averaged elementwise addition of the word embeddings of the words whose POS-tags belong to the **entity set** {'NN': singular noun, 'NNS': plural noun, 'NNP': singular proper noun, 'NNPS': plural proper noun, 'PRP': personal noun}; modification and action components are composed by the same operation, but the **modification set** is {'DT': determiner, 'PDT': predeterminer, 'JJ': adjective, 'JJR': comparative adjective, 'JJS': superlative adjective, 'PRP$': possessive pronoun, 'POS': possessive

ending, ‘EX’: existential there, ‘MD’: modal, ‘RB’: adverb, ‘RBR’: comparative adverb, ‘RBS’: superlative adverb, ‘RP’: particle, ‘TO’: to}, and the **action set** is {‘VB’: base form verb, ‘VBD’: past tense verb, ‘VBG’: present particle verb, ‘VBN’: past participle verb, ‘VBP’: present verb, ‘VBZ’: third person present verb}. For example, if the expression “He likes sea” has three word embeddings (0.1, 0.1, 0.1) for “he”, (0.2, 0.2, 0.2) for “likes”, and (0.3, 0.3, 0.3) for “sea”, based on the operation, the entity component is {(0.1, 0.1, 0.1) + (0.3, 0.3, 0.3)} / 2 = (0.2, 0.2, 0.2); the action component is (0.2, 0.2, 0.2) / 1 = (0.2, 0.2, 0.2); the modification component is (0, 0, 0). If the word embedding of a word does not exist in the pretrained model, our strategy is to skip it.

4.3 Components Weighting

The weighting scheme for five components is based on assumption, observation, and reverse strategy, the deatils of which are described in the paragraphs of this subsection. All descriptions of the following paragraphs in this subsection except last paragraph are related to the weighting scheme for paraphrase sentence pairs of the training dataset. The weighting scheme for non-paraphrase sentence pairs is performed in an opposite way to paraphrase. Summarized information are presented in Table 3.

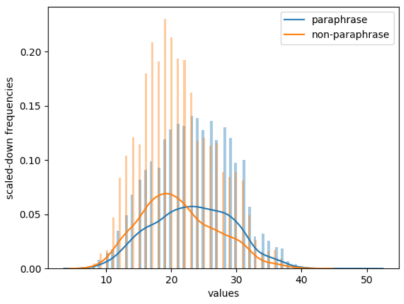


Figure 3: Histogram of sentence length of all sentences belonging to the training dataset; paraphrase sentences have higher frequencies than non-paraphrase sentences when sentence length is no less than 23 tokens.

Three are four observations based on the MSRP’s characteristics, and one of them is Jaccard distance whose effectiveness on semantic similarity is shown in the preliminary experiment. We used it to weight one of the reward components, and adopted 0.6 as threshold – if a sentence pair’s Jaccard distance is no greater than 0.6, a new vector (reward component) is created with length 300 (same as the length of pretrained word embeddings) whose elements are all assigned value of 1 as a strong reward; value of 0.2 is assigned to the elements of a new created vector as a small reward if a pair’s Jaccard distance is greater than 0.6.

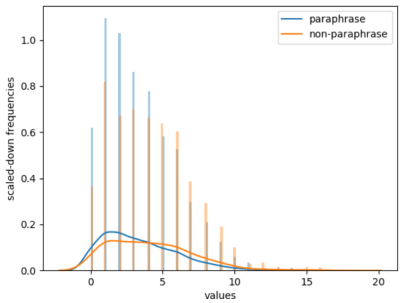


Figure 4: Histogram of absolute difference of sentence length of the sentence pairs; when absolute difference is no greater than 5, paraphrase has higher frequencies.

As for weighting the second reward component, it is based on two observations (see Figure 3 and 4). Again a new vector is created with length 300 as a reward component. Based on the observation of frequencies, if sentence length is no less than 23 or absolute difference of sentence length is no greater than 5, value of 1 is assigned; if sentence length is less than 23 or absolute difference of sentence length is greater than 5, value of 0.2 is assigned.

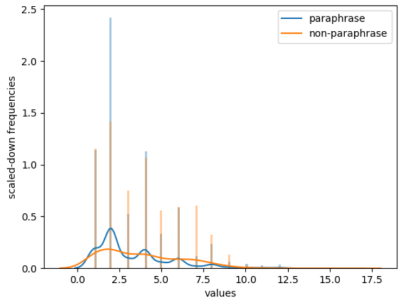


Figure 5: Histogram of digit sums of the sentence pairs; when sum is no greater than 4, the paraphrase sentence pairs have higher frequencies; there is an exception when sum is 3 but difference is not big.

The weighting scheme for the three word-embedding components is based on the combination of assumption (conclusion is not drawn from analysis on the MSRP corpus) and observation. Figure 5 shows digit sums of the sentence pairs (only the digit sums whose values exceed 0 are counted: 1249 paraphrase sentence pairs and 745 non-paraphrase sentence pairs). Below describes a paraphrase sentence pair of the MSRP corpus, where the digit sum (**bold** part in the sentences) is 2, the sentence length for S1 is 16 and 18 for S2, and the absolute difference of sentence length is 2.

	Paraphrase Weighting	Non-paraphrase Weighting

Entity	if digit-sum <= 4:
	Entity = 1.5 * entity component
	else:
	Entity = 0.5 * entity component
		if digit-sum >4:
		Entity = 1.5 * entity component
		else:
		Entity = 0.5 * entity component

Modi.	Modi. = 0.5 * modification component	Modi. = 1.5 * modification component
Action	Action = 1.5 * action component	Action = 0.5 * action component

RF1	if sent-len >= 23 or abs-sent-len-diff <= 5:
	RF1 = new a vector [for 1.0 in range(300)]
	else:
	RF1 = new a vector [for 0.2 in range(300)]
		if sent-len <23 or abs-sent-len-diff >5:
		RF1 = new a vector [for 1.0 in range(300)]
		else:
		RF1 = new a vector [for 0.2 in range(300)]

RF2	if Jaccard-distance <= 0.6:
	RF2 = new a vector [for 1.0 in range(300)]
	else:
	RF2 = new a vector [for 0.2 in range(300)]
		if Jaccard-distance >0.6:
		RF2 = new a vector [for 1.0 in range(300)]
		else:
		RF2 = new a vector [for 0.2 in range(300)]

Table 3: Weighting schemes.

S1 "Trading in Loral was halted yesterday; the shares closed on Monday at $**3.01**."

S2 "The New York Stock Exchange suspended trading yesterday in Loral, which closed at $**3.01** Friday."

The weighting scheme for the three word-embedding components is mainly based on assumption. Below describes two examples of sentence pairs that do not belong to the MSRP corpus.

S1 "The beach is *very amazing*."

S2 "The beach is *fabulous*."

As we can see in the first example, adjective or adverb matters to some extent in terms of semantic similarity. However, in the next example, their roles are less significant (two sentences below are **identical** without them).

S1 "*I like it* very much."

S2 "*I* can't *like it* any more."

In the second example, noun and verb play significant role in terms of semantic similarity while adjective or adverb would better be omitted. Since noun and verb also matter to some extent in the first example, our assumption-based strategy is to give more weights to Entity and Action, and less to Modification. Therefore, to weight three word-embedding components is Entity = 1.5 * entity component, Modification = 0.5 * modification component, and Action = 1.5 * action component; for example, 1.5 * (0.5, 0.5, 0.5) = (0.75, 0.75, 0.75).

Nevertheless, as shown in Table 3, we also take into account the digit sum when it comes to weighting entity components. When digit sum is greater than 4, the weight for entity component is 0.5, because we assume when there are too many digits appear in both sentences, they tend to blur the meaning of the words belonging to the entity set.

Aforementioned assumptions could be plausible but not universal. Thus, for holistic consideration, in order to complement our assumption, the weighting scheme for the three word-embedding components of non-paraphrase sentence pairs is based on reverse strategy, and the weighting scheme for the two reward factors are also performed in an opposite way.

4.4 Neural Network

We trained two spaces (projection matrices) by mapping the concatenation of the five weighted components of a sentence to its vector representation of latent semantics: paraphrase space and non-paraphrase space. The hyperparameters of the neural network is illustrated in Table 4.

Items	Settings
Activation function	tanh
Input length	1,500
Output length	100
Loss function	MSE
Epochs	600
Learning rate	0.0005

Table 4: Neural network hyperparameters.

5 Features

All the features except the basic features are summarized in Table 5. Details are described in the following subsections.

5.1 Basic Features (BFEAT)

Based on the two trained spaces, when we generate the basic features, as shown in Figure 6, two sentences of any pair can respectively get two concatenated components based on paraphrase and non-paraphrase weighting schemes, and then obtain their latent semantics after two concatenated components being projected by two spaces: *Lp(S1)* and *Lp(S2)* are the latent semantics of a sentence pair projected from the paraphrase space; *Lnp(S1)* and *Lnp(S2)* are the latent semantics of a sentence pair projected from the non-paraphrase space, based on which, BFEAT is generated by the concatenation of four parts: *Lp(S1)* + *Lp(S2)*, |*Lp(S1)* - *Lp(S2)*| , *Lnp(S1)* + *Lnp(S2)*, |*Lnp(S1)* - *Lnp(S2)*| , and total length is 400. The concatenation is inspired by the work of (Ji and Eisenstein, 2013).

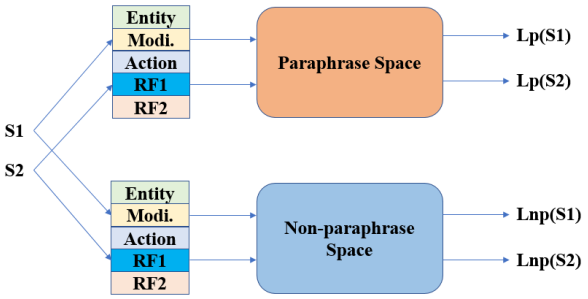


Figure 6: Latent semantics projected by two spaces.

5.2 Distance-based Features (FEAT1)

The Levenshtein edit distance is operation cost (insertion, replacement, and deletion) between two strings; e.g. the edit distance between ”apple” and ”apples” is 1, because there is only one operation of inserting/deleting ‘s’ in the first/second string. Given S1 and S2 of a sentence pair, we treat them as long strings, and measure their edit distance as a consideration of literal semantic distance.

The Chebyshev distance (Lp) is the Chebyshev distance between *Lp(S1)* and *Lp(S2)*. Likewise, the Chebyshev distance (Lnp) is for *Lnp(S1)* and *Lnp(S2)*. Chebyshev distance is described in Equation 2, given two n-dimensional vectors with their elements x1 and x2 respectively $(1 \leqslant i \leqslant n)$. This distance metric is used to measure latent semantic distance between two sentences.

FEAT1
Edit distance
Chebyshev distance (Lp)
Chebyshev distance (Lnp)
Averaged Euclidean distance
Absolute difference of Euclidean distance
FEAT2
Absolute digit difference
Absolute entity difference
Absolute action difference
FEAT3 (Wan et al., 2006)
Unigram recall
Unigram precision
Bigram recall
Bigram precision
Trigram recall
Trigram precision
BLEU recall
BLEU precision
Absolute difference of sentence length

Table 5: All the features except basic features.

$$Cdistance(\vec{v_1}, \vec{v_2}) = max(|x1_i - x2_i|) \quad (2)$$

Two Euclidean distances (see Equation 3), are obtained from *Lp(S1)* $\Rightarrow$ *Lnp(S1)* and *Lp(S2)* $\Rightarrow$ *Lnp(S2)*. After averaging their sum by 2, we obtain the averaged Euclidean distance. By their absolute difference, the absolute difference of Euclidean distance is obtained as well. We consider them as a geometric distance between the latent semantics projected by the two spaces.

$$Edistance(\vec{v_1}, \vec{v_2}) = \sqrt{\sum_{i=1}^{n}(|x1_i - x2_i|)^2} \quad (3)$$

5.3 Distance-contributing Features (FEAT2)

To explore what can be the contributing factors to the distances mentioned in the previous subsection, three distance-contributing features are created. Below describes an example from the MSRP corpus.

S1 “The *notification* **was** first **reported** *Friday* by *MSNBC*.”

S2 “*MSNBC.com* first **reported** the *CIA request* on *Friday*.”

For this pair of sentences, absolute digit difference is 0; *absolute entity difference* is 1; **absolute action difference** is 1.

SVM Input	C	Acc.(%)	Prec.(%)	Rec.(%)	F(%)
BFEAT	0.08	72.75	72.79	**94.25**	82.14
BFEAT + FEAT1	0.06	74.49	75.23	91.89	82.73
BFEAT + FEAT1 + FEAT2	0.1	75.13	76.59	90.15	82.82
BFEAT + FEAT1 + FEAT3	0.07	75.07	76.3	90.67	82.87
BFEAT + FEAT1 + FEAT2 + FEAT3	0.05	**76.64**	**77.56**	91.28	**83.86**
BFEAT + FEAT3	0.04	74.9	74.76	93.98	83.28
Published Works	**Acc.(%)**	**F(%)**			
(Mihalcea et al., 2006) Baseline	65.4	75.3			
(Hu et al., 2014)	69.9	80.9			
(Mihalcea et al., 2006)	70.3	81.3			
(Guo and Diab, 2012)	71.5	not reported			
(Blacoe and Lapata, 2012)	73.0	82.3			
(Wan et al., 2006)	75.6	83.0			
(Conneau et al., 2017)	76.2	83.1			
(Socher et al., 2011)	76.8	83.6			
(He et al., 2015)	78.6	84.7			
(Cheng and Kartsaklis, 2015)	78.6	85.3			
(Ji and Eisenstein, 2013)	**80.4**	**85.9**			

Table 6: Results of paraphrase identification.

5.4 Fine-grained Features (FEAT3)

Table 5 also illustrates some fine-grained features proposed by (Wan et al., 2006), which were also utilized by (Ji and Eisenstein, 2013). We consider them as another set of distance-contributing features. In our work, we removed dependency relation and tree edit distance as we were not able to obtain permission to use Connexor parser[5] as of writing, but added trigram recall and precision.

6 Paraphrase Identification

The results of paraphrase identification based on our features and previously published works are summarized in Table 6. For classification, we choose Support Vector Machine with a linear kernel provided again by Scikit-learn tool. The tuning on parameter C is contained in interval [0.01, 0.1], and increment is 0.01. When we combine all the feature sets together, the highest accuracy 76.64% associated with F-score 83.86% is achieved. When the basic features are only combined with the borrowed features, the result is comparatively low but outperforms half of the previously published works.

The published works listed by us for comparison utilize either distributional methods or complex neural networks (marked with grey background). We also list (Wan et al., 2006) 's work for the respect of their fine-grained features. (Socher et al., 2011) whose method is recursive autoencoder with dynamic pooling achieved almost the same performance as ours. Except (Ji and Eisenstein, 2013), our proposed method outperforms the rest of distributional methods, and two complex neural networks: (Conneau et al., 2017) (BiLSTM) and (Hu et al., 2014) (Convolutional NN) , but still lags behind (He et al., 2015) (Convolutional NN) and (Cheng and Kartsaklis, 2015) (Recursive NN). Nevertheless, our approach consumes little system resources. As for (Ji and Eisenstein, 2013), they used transductive learning (Gammerman et al., 1998) to perform matrix factorization on (unlabeled) test data and training data. In contrast, test data are completely disjoint from training data in our work.

7 Conclusion and Future Work

In this paper, we propose a novel method to project weighted word embeddings on latent space as an approach to improving distributional semantic similarity of sentences. Our proposed method has achieved better performance on Microsoft Research Paraphrase Corpus than many published methods. For our future work, we plan to explore our proposed method by conducting experiments on other semantic similarity tasks as well as trying different weighting criteria, word embedding models, and POS-tag combinations.

[5]https://www.connexor.com/nlplib/?q=demo/syntax

References

William Blacoe and Mirella Lapata. 2012. A comparison of vector-based representations for semantic composition. In *Proceedings of the 2012 joint conference on empirical methods in natural language processing and computational natural language learning*, pages 546–556. Association for Computational Linguistics.

David M Blei, Andrew Y Ng, and Michael I Jordan. 2003. Latent dirichlet allocation. *Journal of machine Learning research*, 3(Jan):993–1022.

Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146.

Jianpeng Cheng and Dimitri Kartsaklis. 2015. Syntax-aware multi-sense word embeddings for deep compositional models of meaning. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1531–1542, Lisbon, Portugal. Association for Computational Linguistics.

Kenneth Ward Church and Patrick Hanks. 1990. Word association norms, mutual information, and lexicography. *Computational linguistics*, 16(1):22–29.

Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. 2017. Supervised learning of universal sentence representations from natural language inference data. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, Copenhagen, Denmark. Association for Computational Linguistics.

Scott Deerwester, Susan T Dumais, George W Furnas, Thomas K Landauer, and Richard Harshman. 1990. Indexing by latent semantic analysis. *Journal of the American society for information science*, 41(6):391–407.

Bill Dolan, Chris Quirk, and Chris Brockett. 2004. Unsupervised construction of large paraphrase corpora: Exploiting massively parallel news sources. In *COLING 2004: Proceedings of the 20th International Conference on Computational Linguistics*, pages 350–356, Geneva, Switzerland. COLING.

Alexander Gammerman, Volodya Vovk, and Vladimir Vapnik. 1998. Learning by transduction. In *Proceedings of the Fourteenth conference on Uncertainty in artificial intelligence*, pages 148–155. Morgan Kaufmann Publishers Inc.

Weiwei Guo and Mona Diab. 2012. Modeling sentences in the latent space. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 864–872, Jeju Island, Korea. Association for Computational Linguistics.

Hua He, Kevin Gimpel, and Jimmy Lin. 2015. Multi-perspective sentence similarity modeling with convolutional neural networks. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1576–1586.

Baotian Hu, Zhengdong Lu, Hang Li, and Qingcai Chen. 2014. Convolutional neural network architectures for matching natural language sentences. In *Advances in neural information processing systems*, pages 2042–2050.

Paul Jaccard. 1912. The distribution of the flora in the alpine zone. 1. *New phytologist*, 11(2):37–50.

Yangfeng Ji and Jacob Eisenstein. 2013. Discriminative improvements to distributional sentence similarity. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 891–896.

Thomas K Landauer and Susan T Dumais. 1997. A solution to plato's problem: The latent semantic analysis theory of acquisition, induction, and representation of knowledge. *Psychological review*, 104(2):211.

Daniel D Lee and H Sebastian Seung. 1999. Learning the parts of objects by non-negative matrix factorization. *Nature*, 401(6755):788.

Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK.

Rada Mihalcea, Courtney Corley, Carlo Strapparava, et al. 2006. Corpus-based and knowledge-based measures of text semantic similarity. In *Aaai*, volume 6, pages 775–780.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pages 3111–3119.

George A. Miller. 1995. Wordnet: A lexical database for english. *Commun. ACM*, 38(11):39–41.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in python. *Journal of machine learning research*, 12(Oct):2825–2830.

Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543.

G. Salton. 1971. *The SMART Retrieval System—Experiments in Automatic Document Processing*. Prentice-Hall, Inc., Upper Saddle River, NJ, USA.

Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information processing & management*, 24(5):513–523.

Amit Singhal, Gerard Salton, Mandar Mitra, and Chris Buckley. 1996. Document length normalization. *Information Processing & Management*, 32(5):619–633.

Richard Socher, Eric H Huang, Jeffrey Pennin, Christopher D Manning, and Andrew Y Ng. 2011. Dynamic pooling and unfolding recursive autoencoders for paraphrase detection. In *Advances in neural information processing systems*, pages 801–809.

Peter D Turney and Patrick Pantel. 2010. From frequency to meaning: Vector space models of semantics. *Journal of artificial intelligence research*, 37:141–188.

Stephen Wan, Mark Dras, Robert Dale, and Cécile Paris. 2006. Using dependency-based features to take the'para-farce'out of paraphrase. In *Proceedings of the Australasian Language Technology Workshop 2006*, pages 131–138.

Wei Xu, Alan Ritter, and Ralph Grishman. 2013. Gathering and generating paraphrases from twitter with application to normalization. In *Proceedings of the sixth workshop on building and using comparable corpora*, pages 121–128.